

Data Speaks. Can You Hear It?

Honours Workshop — Statistical Inference with Python

Tidy → Group → Predict · ~40 minutes

TIDY	GROUP	PREDICT
Real data is broken	Patterns hide in plain sight	The past knows the future
<i>You fix it</i>	<i>You find them</i>	<i>You extract it</i>

Q: You're handed a CSV. Column names have spaces and dollar signs. Some cells are blank. Three rows look identical. One salary says **\$856,580** when the max should be ~\$150K. What do you do **first**?

Discuss with your neighbour (30 sec). Then share one word with the room.

Your Datasets

	salary_messy.csv	customer_segments.csv
Rows	504 (messy)	400
Cols	8 (2 are junk)	6
Used for	Tidying + Prediction	Grouping
Question	What drives salary?	Who are your customers?

Both synthetic, built with known true DGPs. Every t-statistic is verifiable.

Quick Setup

```
pip install pandas numpy matplotlib seaborn statsmodels scikit-learn
```

Check it works:

```
import pandas as pd  
print(pd.__version__) # ≥ 2.0
```

Q: Raise your hand when you see a version number. Don't move on until everyone has it.

PART 1

TIDYING

10 min · 6 slides

"The data is broken. Fix it."

Load the Mess

```
import pandas as pd
import numpy as np

df = pd.read_csv("salary_messy.csv")
print(df.info())
```

Column	Non-Null	Problem?
Annual Salary (\$)	479 / 504	25 missing
Manager? (1=Yes)	504	Ugly name
Department_Code	504	FIN:::NCE
Education_Numeric	504	Redundant

Q: 504 rows when there should be 500. What's going on?
Discuss: Think for 10 seconds, then pair-share.

Fix 1: Rename Columns

```
df = df.rename(columns={
    'Annual Salary ($)': 'salary',
    'Years Experience': 'experience',
    'Education Level': 'education',
    'Manager? (1=Yes)': 'is_manager',
    'Hours/Week': 'hours',
    'Department': 'department',
    'Department_Code': 'dept_code',
    'Education_Numeric': 'edu_code',
})
```

Why rename? No spaces → `df.salary` works. No special chars → formulas don't break. Lowercase → no SHIFT key accidents. Every analyst does this within **30 seconds** of opening a stranger's file.

Fix 2 & 3: Missing + Duplicates

```
# 25 missing salaries
print(f"Missing: {df['salary'].isna().sum()}")
df = df.dropna(subset=['salary']) # 479 rows

# 3 duplicate rows
print(f"Duplicates: {df.duplicated().sum()}")
df = df.drop_duplicates() # 476 rows
```

Step	Rows	What happened
Original	504	Messy file
<code>dropna()</code>	479	–25 rows with blank salary
<code>drop_duplicates()</code>	476	–3 copied rows

Q: How do duplicates happen? (*Export errors, merge bugs, human copy-paste.*)

Discuss: Take 15 sec, then call out your best guess.

Fix 4: The Outlier

```
import matplotlib.pyplot as plt
plt.boxplot(df['salary'], vert=False)
plt.show()
```

One point at **\$856,580** — everything else below ~\$156K.

```
q99 = df['salary'].quantile(0.99) # $151,381
df = df[df['salary'] ≤ q99 * 3] # → 475 rows
```

5.7× the 99th percentile. Data entry error (extra zero) or the CEO's salary. **Context matters** — always verify outliers against domain knowledge.

Fix 5: Split & Save

```
# 'FIN:::NCE' → two columns
df[['dept_abbr', 'dept_suffix']] = (
    df['dept_code'].str.split(':::', expand=True)
)

# Drop originals + redundant columns
df = df.drop(columns=['dept_code', 'dept_suffix', 'edu_code'])

df.to_csv('salary_clean.csv', index=False)
```

	Before	After
Rows	504	475
Columns	8 (messy)	7 (clean)
Missing	25	0
Duplicates	3	0
Outliers	1 (\$857K)	0

Tidying: Why It Matters

Q: What if you skip tidying?

- Run regression with the **\$857K outlier** → experience coef inflates by ~\$300/year
- Don't remove **duplicates** → p-values look more significant than they are
- Leave **bad column names** → 5-minute debugging session on a KeyError typo

Tidying is not busywork. Tidying is the difference between a finding and a mistake.

Discuss: Any questions before we move on?

End of Part 1 Recap

What We Just Did

Problem	Solution	Rows affected
Ugly column names	<code>df.rename()</code>	All 8 columns
25 missing salaries	<code>df.dropna()</code>	-25
3 duplicate rows	<code>df.drop_duplicates()</code>	-3
1 extreme outlier	Quantile filter	-1
Concatenated codes	<code>str.split(':::')</code>	Split into 2 cols
Redundant columns	<code>df.drop()</code>	-2 columns

Clean result: 475 rows · 7 columns · 0 problems

PART 2

GROUPING

15 min · 8 slides

"Patterns hide in plain sight. You find them."

Customer Segments

```
df = pd.read_csv("customer_segments.csv")  
print(f"{len(df)} customers")
```

annual_spend	visits/mo	basket	loyalty	segment_label
\$11,926	4.0	179.7	4.0 yr	Whale
\$5,728	14.2	71.0	5.0 yr	Premium Loyalist
\$1,389	9.9	22.1	1.1 yr	Budget Shopper

400 customers. 4 features. 3 segments. *But in reality, you don't have the `segment_label` column — you discover the groups yourself.*

groupby() — When You Have Labels

```
df.groupby('segment_label')[
    ['annual_spend', 'visit_frequency_month',
     'avg_basket_size', 'loyalty_years']
].mean().round(1)
```

Segment	Spend/yr	Visits/mo	Basket	Loyalty
Budget Shopper	\$1,239	14.2	21.9	2.7 yr
Premium Loyalist	\$4,454	9.8	53.5	7.3 yr
Whale	\$9,377	6.0	117.1	5.1 yr

Q: Discuss with your neighbour (30 sec): If you had ONE billboard ad, which segment do you target and **why**? Defend your answer with a number from this table.

Q: What If You DON'T Have Labels?

Three Hidden Groups in the Feature Space

Feature	Budget Shopper	Premium Loyalist	Whale
Annual spend	Low (\$1,200)	Medium (\$4,500)	High (\$9,500)
Visit frequency	High (14/mo)	Medium (10/mo)	Low (6/mo)
Basket size	Small (\$22)	Medium (\$55)	Large (\$120)
Loyalty	Short (2.5 yr)	Long (7.0 yr)	Medium (5.0 yr)
Strategy	Volume-driven	Relationship-driven	Premium-driven

Q: Before we run clustering — guess how many groups exist in this data just from this table.

Discuss: Write your answer on your hand. Show me on 3.

Run K-Means (K=3)

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X = df[['annual_spend', 'visit_frequency_month',
        'avg_basket_size', 'loyalty_years']]

X_scaled = StandardScaler().fit_transform(X) # SCALE is critical!

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X_scaled)

print(pd.crosstab(df['segment_label'], df['cluster']))
```

True Segment	Cluster 0	Cluster 1	Cluster 2
Budget Shopper	0	159	1
Premium Loyalist	1	7	132
Whale	92	0	8

The algorithm found the same groups WITHOUT seeing the labels.

How Many Clusters? The Elbow

```
inertias = []  
for k in range(1, 9):  
    km = KMeans(n_clusters=k, random_state=42, n_init=10)  
    km.fit(X_scaled)  
    inertias.append(km.inertia_)
```

K	1	2	3	4	5	6	7	8
Inertia	1,600	832	498	402	354	312	289	267

1,600 → 832 = -768 | 832 → 498 = -334 | 498 → 402 = -96

The elbow is at **K = 3**

Classify New Customers

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, df['segment'], test_size=0.3, random_state=42
)

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)

print(f"Accuracy: {clf.score(X_test, y_test):.1%}")
```

95.8% accuracy

Baseline (random guess with 3 equal classes): **33%**

Grouping: Your Toolkit

Approach	Type	Question Answered
<code>df.groupby().mean()</code>	Supervised summary	<i>What do my known groups look like?</i>
<code>KMeans(n_clusters=3)</code>	Unsupervised clustering	<i>What groups EXIST in my data?</i>
Elbow method	Model selection	<i>How many groups are there?</i>
<code>LogisticRegression()</code>	Supervised classification	<i>Which group is this NEW customer?</i>

Q: FINAL GROUPING QUESTION: You're the marketing director. Budget covers ONE email campaign. Which segment — and defend with a number from the groupby table.

Discuss: 30 seconds to decide. Then we compare answers across the room.

End of Part 2 Recap

What We Just Did

Concept	Code / Method	Outcome
Labeled groups	<code>df.groupby().mean()</code>	Segment profiles
No labels → K-Means	<code>KMeans(n_clusters=3)</code>	Found 3 true segments
Choosing K	Elbow method	K=3 confirmed
Classify new customers	<code>LogisticRegression()</code>	95.8% accuracy

Three segments: Budget Shopper · Premium Loyalist · Whale

PART 3

PREDICTION

15 min · 9 slides

"The past knows the future. You extract it."

Explore: Experience vs Salary

```
df = pd.read_csv("salary_clean.csv")
print(df.describe())
```

	salary	experience	is_manager	hours
mean	\$97,077	18.1 yr	19%	38.0 hr
std	\$25,355	10.1 yr	39%	6.0 hr
range	\$33K-\$156K	0.2-34.8 yr	0-1	20-60 hr

Education: 201 Bachelor's · 144 High School · 130 Master's

```
corr = df['experience'].corr(df['salary'])
print(f"Correlation: {corr:.3f}") # → 0.888
```

Q: Correlation is 0.888. Is that "good"? What does it actually **MEAN**?
Discuss: Think for 10 seconds. Then write your answer in one sentence.

Your First Regression

```
import statsmodels.formula.api as smf

model1 = smf.ols('salary ~ experience', data=df).fit()
print(model1.summary())
```

```
OLS Regression Results


```

	coef	std err	t	P> t
Intercept	5.74e+04	1218.327	47.146	0.000
experience	2227.98	53.050	**42.000**	**0.000**

```


R-squared:  **0.789**
```

Q: `salary ~ experience` — what does the `~` mean in plain English?

Discuss: "Salary is predicted **BY** experience" or "Salary depends **ON** experience."

How to Read a Regression Table

Only four numbers matter. Everything else: greyed out.

	What	Value	Meaning
	coef	+\$2,228	Each year of experience → +\$2,228
	P> t 	< 0.001	Virtually certain this is real
	t-statistic	42.0	42 standard errors from zero — enormous
	R²	0.789	Experience explains 78.9% of salary

95% CI: true effect between **\$2,124** and **\$2,333** per year of experience.

The t-statistic

$$t = \text{coefficient} \div \text{standard error} = \$2,228 \div \$53 = 42.0$$

What t-values Mean

t-value	p ≈	Interpretation	Verdict
0.5	0.62	Coefficient is ~half its SE	Not significant — noise
1.5	0.13	Coefficient is 1.5× its SE	Might be real, can't be sure
2.0	0.05	Coefficient is 2× its SE	Significant (conventional)
3.0	0.003	Coefficient is 3× its SE	Highly significant
42.0	6.8×10^{-162}	Coefficient is 42× its SE	Overwhelming

Rule of thumb: $|t| > 2 \rightarrow$ significant at the 5% level (for large samples). Our $t=42$ is off the charts.

Add More Variables

```
model2 = smf.ols(  
    'salary ~ experience + education + is_manager + hours',  
    data=df  
) .fit()
```

Variable	Coef	t	p	Interpretation
experience	+\$2,183	57.2	<0.001	Each year → +\$2,183
education[T.High School]	-\$5,123	-5.6	<0.001	HS grads: -\$5,123 vs Bachelor's
education[T.Master]	+\$7,071	7.5	<0.001	Master's: +\$7,071 vs Bachelor's
is_manager	+\$16,267	16.5	<0.001	Managers: +\$16,267
hours	+\$287	4.7	<0.001	Each extra hour → +\$287
R ²	0.892	—	—	89.2% explained

Q: The experience coef was **\$2,228** in Model 1. Now it's **\$2,183**. Why did it change?

Discuss: Take 20 seconds — chat with your neighbour. Use the word 'omitted variable bias'.

What About Department?

```
model3 = smf.ols(  
    'salary ~ experience + education + is_manager + hours + C(department)',  
    data=df  
).fit()
```

Variable	Coef	t	p	Interpretation
experience	+\$2,167	63.2	<0.001	Each year → +\$2,167
education[T.High School]	-\$5,088	-6.3	<0.001	vs Bachelor's
education[T.Master]	+\$7,482	8.9	<0.001	vs Bachelor's
is_manager	+\$17,022	19.2	<0.001	+\$17K for managers
hours	+\$284	5.3	<0.001	Per extra hour
department[T.Finance]	+\$3,931	4.1	<0.001	vs Engineering
department[T.Marketing]	-\$5,956	-6.0	<0.001	vs Engineering
department[T.Operations]	-\$3,957	-4.0	<0.001	vs Engineering
R²	0.915	—	—	91.5% explained

Department effects relative to **Engineering** (reference). Finance premium, Marketing penalty — both survive controls.

The Bleeding Edge: Beyond OLS

	OLS	Random Forest	XGBoost
Interpretability			
Predictive power			
Assumptions	Linearity, normality	None	None
Output	coef, p, t, R ² , CI	Feature importance	Feature importance
Question	<i>Why?</i>	<i>What?</i>	<i>What, accurately?</i>

Q: If XGBoost predicts better, why ever use OLS?

"Salary goes up by \$2,167/year and we're 99.999% confident" — you can defend that statement to a boss, a journal, or a court. "The black box says so" — you can't.

Discuss: Any thoughts on when you'd prefer interpretability over accuracy?

Your 8-Line Toolkit

```
# TIDY
df = pd.read_csv("data.csv")
df = df.rename(columns={'Bad Name': 'good_name'})
df = df.dropna().drop_duplicates()

# GROUP
df.groupby('category').mean()
KMeans(n_clusters=3).fit_predict(X_scaled)
LogisticRegression().fit(X_train, y_train)

# PREDICT
sns.regplot(x='x', y='y', data=df)
model = smf.ols('y ~ x1 + x2 + C(category)', data=df).fit()
print(model.summary()) # coef, p, t, R² – your 4 numbers
```

Screenshot this. Works on **ANY** dataset with a numeric target.

Data Speaks. Now You Do Too.

Find a CSV — Kaggle, CBS StatLine, Eurostat

Try `RandomForestRegressor` — compare to OLS

Add an instrument — you have IV for your thesis

Files: `salary_clean.csv` · `customer_segments.csv` · `generate_datasets.py` · This deck